Sliding Window Aggregation

(4 points.) To start, run `python new-homework.py [language] swag`

Background. Streaming processing of data in real time (aka "online") is a significant practical problem, with a wide range of applications. Here, we consider the important special case of computing aggregations of input values over "sliding windows" in the data stream. This problem has been the subject of extensive research because it can be challenging to do efficiently and because it supports a variety of interesting operations and use cases.

Suppose we have a stream of data $x_0, x_1, x_2, \ldots$, where all x_i have a common type. The stream may terminate after a finite number of values or it may continue ad infinitum. Our goal is to compute a new stream $y_0, y_1, y_2, \ldots$ from the input stream by *aggregating over sliding windows of a specified size* w with:

$$\begin{aligned} y_0 &= f(x_0, x_1, \ldots, x_{w-1}) \\ y_1 &= f(x_1, x_2, \ldots, x_w) \\ y_2 &= f(x_2, x_3, \ldots, x_{w+1}) \\ &\vdots \end{aligned}$$

for some fixed function f.

We will specialize this setup in two steps. First, we assume that the input values have type a, i.e., $x_i : a$ for all i, and let $\oplus \colon a \times a \longrightarrow a$ be an *associative* binary operator (i.e., $(x \oplus y) \oplus z = x \oplus (y \oplus z)$), though possibly not commutative or invertible. Define $f \colon a^w \longrightarrow b$ by

$$f(s_1, s_2, \ldots, s_w) = u \oplus s_1 \oplus s_2 \oplus \cdots \oplus s_w$$

for a fixed starting value $u : a$.

Second, we generalizes the first step by *annotating* the input values before aggregating them and then transforming the aggregated value back to the original type. We assume that the input values have type v, i.e., $x_i : v$ for all i, and that we have functions

$$\begin{aligned} &\mathsf{annotate} \colon v \longrightarrow a \\ &\mathsf{finalize} \colon a \longrightarrow v \\ &\oplus \colon a \times a \longrightarrow a, \end{aligned}$$

where again $\oplus$ is associative. Now, define $f \colon v^w \longrightarrow b$ by

$$f(t_1, t_2, \ldots, t_w) = \mathsf{finalize}\Big(u \oplus \mathsf{annotate}(t_1) \oplus \cdots \oplus \mathsf{annotate}(t_w)\Big),$$

for the same fixed starting value $u : a$.

We will consider a variety of assumptions about the type a and the binary operator $\oplus$, as shown in Table 1. A variety of interesting aggregation operations fall under this framework, including for instance:

- "Sum-like" (Monoid or Group): sum, count, average, standard deviation, ...
- Collection (Monoid): gather, concatenate strings, merge dicts, ith occurrence, ...
- "Median-like" (Ordered Monoid): median, ith smallest, quantiles, ...

	Associative	Identity	Commutative	Invertible	Ordered
Semigroup	✓	✗	✗	✗	maybe
Monoid	✓	✓	✗	✗	maybe
Commutative Monoid	✓	✓	✓	✗	maybe
Group	✓	✓	maybe	✓	maybe

Table 1: Algebraic assumptions about the aggregation operator $\oplus$ and type a.

- "Max-like" (Ordered Monoid or Semigroup, maybe Commutative): max, min, argMax, max count or feature, ...
- "Sketch-like" (Commutative Monoid): Bloom filter, CountMin, HyperLogLog, ...

(The term "sketch" here refers to a compact, approximate summary of a data set, often probabilistic.) These different assumptions lead to different algorithms and data structures that vary in optimal performance. The Monoid assumption will be our base case, and we will explore some of the variations as well.

For what follows, you may need to reference the TL1 intro document, and Interlude F may also help.

Exercise #1. Show how to define annotate, formalize, and $\oplus$ in the monoid case so that the aggregation produces

1. the average of values in the window, assuming $a = \mathbb{R}$,
2. the standard deviation of values in the window, assuming $a = \mathbb{R}$, and
3. the simple linear regression coefficients of data in the window, assuming $a = \mathbb{R} \times \mathbb{R}$,

Task. Given a sequence/stream of values of some type, we want to aggregate values in *sliding windows* of specified length into a new sequence/stream. We will implement several versions of a function swag to do this under various assumptions about the aggregation operation. The input stream can be from a data structure in memory, a generator of some sequence, or a source like a file or web server.

The function swag needs to know:

1. the ***associative*** operation,
2. an initial value for the aggregation in each window,
3. the window size to use, and
4. the input stream.

The function returns a new sequence or stream consisting of successive (complete) windows. For our purposes, we will ignore partial windows at the end of the sequence, but you can if you like add an option to include them. The overall goal is to do this aggregation efficiently.

As mentioned earlier, our main case will be the monoid case, where the binary operator is associative and the starting value $u : a$ is the identity element for the monoid. In this case, our goal is to **make only $O(n)$ calls to the binary operation *independent of the window size***. We require that the number of calls to the binary operation is independent of the window size in an *amortized* way, meaning that the number of calls *per input element* is $c + o(1)$ in the window size, for some constant c.

It will help to package up the information about the operator and starting value that we will need

```
data Agg v a = record where
                 op : a -> a -> a
                 init : a
                 annotate : v -> a
                 finalize : a -> v

-- Step 1 setup: Monoidal values and operations
basic : Monoid a => Agg a a
basic = Agg (<>) munit identity identity

-- Step 2 setup: values that can be extended/projected to/from a monoid
annotated : Monoid a => (v -> a) -> (a -> v) -> Agg v a
annotated ann fin = Agg (<>) munit ann fin
```

The TL1 code `Monoid a =>` is a ***constraint***, requiring that type **a** have an implementation of the **Monoid** trait with operator <> and unit `munit`. Notice that basic is defined for all monoids `a`; when we use it, the compiler infers the type `a` and uses the correct operation and unit. In R or Python, we need to construct the basic record we want to use more explicitly.

We will start with a simple version of swag that uses an in-memory sequence for the input. An idealized TL1 type signature for this version of swag looks like

```
swag' : (agg : Agg v a) -> (wsize : PosNat) -> (input : List v) -> List v

swag : Monoid a => PosNat -> List a -> List a
swag = swag' basic
```

We start with the signature of a more general function `swag'`. Its first argument to `swag'` specifies a general aggregator compatible for some annotation type `a`, as in step 2 of "Background." This can be specified in practice in several ways: as an object/record as above, as a trait/interface, or even as multiple arguments. (You can choose how to do this.) The second argument is a positive window size. The **PosNat** requires proof of positivity to construct, and for instance, given a literal number as a the window size, a compiler for TL1 or similar language (including TypeScript) could infer that proof automatically at *compile time*. In a dynamic language like Python or R, we think of this as just an **int** and make a *runtime* check instead.

```
swag' : Agg v a ->  Int -> List v -> List v
swag' agg windowSize input
  | windowSize < 1  = error "swag requires a positive window size"
  | windowSize == 1 = input
  | otherwise       = ?todo  -- we fill in this "hole"
```

The third argument is the input, which we take as a list for the moment. We use `List a` as a general sequence type for input and output, because we can efficiently access its element

one at a time. But you can replace this with any suitable sequence type, such as a vector or array. The function should return a sequence of the same type.

The function `swag` specializes this to step 1 of "Background," where our input is a list of monoidal values and we use the `basic` aggregator consistent with that monoidal type `a`.

In this task, we will implement `swag` directly rather than as a special case of `swag'`, which will come later. In R or Python, this will look like

```python
class Monoid[A](Protocol):
    def mop(self, x: A, y: A) -> A:
        ...
    @property
    def munit(self) -> A:
        ...
@dataclass
class Agg[V, A]:
    op: Callable[[A, A], A]
    init: A
    annotate: Callable[[V], A]
    finalize: Callable[[A], V]

# In the monoid case, we *build* an Agg[A, A] using a value monoid : Monoid[A]
def basic[A](monoid: Monoid[A]) -> Agg[A, A]:
    return Agg(lambda a1, a2: monoid.mop(a1, a2), monoid.munit, identity, identity)

def swag[A](monoid: Monoid[A], wsize: int, input: list[A]) -> list[A]:
    if wsize < 1:
        raise ValueError('...')
    agg: Agg[A, A] = basic(monoid)
    ...   # fill this in here
```

and in R we define

```r
Monoid <- new_class(
  name = "Monoid",
  abstract = TRUE
)
munit <- new_generic("munit", dispatch_args = "monoid")
mdot  <- new_generic("mop",   dispatch_args = "monoid")

Agg <- new_class(
  name = "Agg",
  properties = list(
    op       = class_function,  # type: a -> a -> a
    init     = S7_any,          # type: a
    annotate = class_function,  # type: v -> a
    finalize = class_function   # type: a -> v
  )
```

```
)
basic <- function(monoid) {
    return( Agg(\(a1, a2) mop(monoid, a1, a2), munit(monoid), identity, identity) )
}
swag <- function(monoid, wsize, input) {
    if( wsize < 1 ) stop("...")
    agg <- basic(monoid)
    ...   # fill this in here
```

Task #1. Implement this monoid-sequence version of `swag` using only $O(n)$ calls to the binary aggregation operation independent of the window size (amortized). Try to avoid any dependence on the length of the input sequence and avoid indexing into the sequence. Imagine that the window size and sequence length can be very large, and in the streaming case below, length and indexing will not be options.

To get a feel for the problem, try starting with a naive implementation that can use the binary operation freely. Come talk to me if you get stuck on making this more efficient.

Streams. We will next refine our implementation to use a general *stream* for the input. A stream is an abstraction of (possibly infinite) sequence of data of some type. A simplified expression of the idea is

```
data Stream a r = Terminated r | Active a (Stream a r)
```

A stream of data has either terminated, returning a single value of type `r`, or it is still active yielding the next element of type `a` and the remainder of the data stream. (In practice, our TL1 expression of this type would be more complicated to account for possible side effects, like reading from a file, but we will ignore that for the moment.)

The main operation on streams is pulling off the next value, if it is available, as captured by the function `next`:

```
next : Stream a r -> Either r (a, Stream a r)
next stream = case stream of
                Terminated ret -> Left ret
                Active current rest -> Right (current, rest)
```

This checks the two cases, returning the designated return value or the current value and remaining stream, tagging them appropriately so that the caller can distinguish.

Task #2. Implement a simple `Stream` interface in your language. Nothing fancy is needed. (Feel free to extend yourself here.) A simple approach gives separate implementations for each type of stream (e.g., list, file handle). You need only three basic functions: (i) `makeStream` that takes the source (e.g., list, file handle) and returns data sufficient to operate on as a stream, (ii) `next` as defined above, and (iii) `isActive` that tests a stream to see if it is still active. In a language like Python or R, it may be convenient for your `next` to execute a cleanup action in the Terminated case (e.g., closing file handles).

For example, to make a stream from a Python list, we can define

```
def make_stream(xs : list):
    return xs

def is_active(list_stream) -> bool:
    return len(list_stream) > 0
```

The list itself is the stream. (You can also use generators if that is a familiar idea.) But in R, a stream reading a file might be defined as

```
makeStream <- function(filename) {
    if ( file_exists(filename) ) {
        return( file(filename, "r") )
    }
    stop("A file reading stream requires an existing file")
}

isActive <- isOpen
```

In this case, `next` might close the connection when end of file is read. In R, where only one namespace is allowed per package, you can store these like-named functions in separate named lists and use, e.g., `fileStream$makeStream` or `listStream$next`, and you can make generic functions that extract these from any stream. See me if you have questions about this.

Optional Exercise #2. Sketch out a definition for a function `partition`

```
partition : Int -> Stream a r -> Stream (List a) r
```

in terms of `next`. This should partition a stream into a stream of lists containing successive chunks of the specified size (or less only if there are remaining values at the end of the stream). You can replace the `List a` with any suitable type as before.

Optional Exercise #3. Sketch out how your implementation in either Task #1 or Task #2 would need to change if you assume only a semigroup condition on a and where $u : a$ is not an identity element.

Task #2. Implement a streaming version of `swag` that meets the same conditions as in Task #1. This should not require significant changes to the basic logic of your previous implementation. This version of `swag` has signature

```
swag : Agg a a -> Int -> Stream a Unit -> Stream a Unit
```

where `Unit` is the unit type with one value `()`. Again, you can change the signature to accommodate the monoid as before.

Task #3. Building on one of your implementations of `swag` in Tasks #1 or 2, implement the generalized form of `swag` (what we called `swag'` above), where you use an `annotate` function to convert values of type v to a monoid type a and `finalize` function to convert the aggregate value for each window back to type v. Now, calling it `swag` here

```
swag' : Agg v a -> Int -> Stream v Unit -> Stream v Unit
```

Use this to compute a sliding-window aggregation that computes running simple linear regression coefficients over a suitably sized window.

Stacks, Queues, and Groups. A stack is a data structure in which one can access and remove only the last item to inserted. Think of a stack of trays in a cafeteria. A queue is a data structure in which one enters at the back and exits at the front, like a line at a grocery store. Both of these data structures can be useful in this assignment, so here we review the basics and add some extras for later tasks.

A simple interface for a stack has two functions `push` to insert an item on the top of the stack and `pop` to remove the top element from the stack.

```
push : a -> Stack a -> Stack a
pop : (s : Stack a) -> {proof: isFalse(isEmpty s)} -> (a, Stack a)
```

It is invalid to pop an empty stack; we encode that with the implicit argument `proof` that proofs the stack is non-empty in order to call `pop`. If we make a runtime check instead, we can eschew that argument at the cost of a possible runtime error.

We can give a simple implementation of stacks in terms of list with runtime checks rather than proof objectsP

```
Stack : Type -> Type     -- Stack is an alias for List
Stack = List

push : a -> Stack a -> Stack a
push elt stack = elt :: stack

-- One version with a runtime instead of a compile-time check
pop : Stack a -> (a, Stack a)
pop [] = error "attempt to pop an empty stack"
pop (top :: rest) = (top, rest)

-- A runtime-safe version that requires the caller check
pop Stack a -> Maybe (a, Stack a)
pop [] = None
pop (top :: rest) = Some (top, rest)
```

A *queue* is a data structure analogous to a line at an airport. It has a front and a back. Entries are added at the back and leave from the front. We can build what is sometimes called a *Banker's queue* by combining two stacks, one for the front and one for the back. We enqueue an item by pushing it on the back stack and dequeue an item by popping it off

the front stack. The only wrinkle is that when the front stack is empty, we need to move the elements from the back stack to the front stack. Although this occasional operation is costly, the amortized cost is low; through each element's life cycle on the queue, we touch it exactly three times (enqueue, move, dequeue). So the amortized cost is $O(1)$. If we use group-annotated stacks, we get a group-annotated Banker's queue.

For Task #4, we require something a little extra – a *group-annotated stack*.

```
data GroupStack g a = record where
                        group    : Group g
                        annotate : a -> g
                        total    : g
                        stack    : List a

push : a -> GroupStack g a -> GroupStack g a
push elt (GroupStack { group, annotate, total = g, stack = s }) =
  GroupStack { group
             , annotate
             , total = (annotate elt) `group.op` g
             , stack = elt :: s
             }

pop : GroupStack g a -> Maybe (a, GroupStack g a)
pop (GroupStack {stack=[]}) = None
pop (GroupStack { group, annotate, total=g, stack=(x :: xs) }) =
  Some (GroupStack { group
                   , annotate
                   , total = (group.inverse (annotate x)) `group.op` g
                   , stack = xs
                   })
```

This stack maintains a group value that represents the total (under the group operation) of all the elements currently on the stack. To push, we simply add in the annotation of the new element, and to pop, we use the inverse of the element on the top of the stack.

Optional Exercise #4. This trick with the inverse will not work with a monoid. Sketch the implementation of a *monoidally-annotated* stack. Instead of a single annotation like `total`, you will need to annotate each element on the stack.

Optional Exercise #5. Sketch an interface for a group-annotated Banker's queue. If you like, implement it.

Task #4. Implement a streaming version of `swag` that assumes that a is a group. The signature will be

```
swag : Group a => Agg a a -> Int -> Stream a () -> Stream a ()
```

or similarly with `List a` or the like. You should be able to make this version more efficient than the just monoid case. You might find it helpful to use a group-annotated Banker's queue or some other queue-style data structure.

Example. Several examples will illustrate the idea of sliding-window aggregation.

- `swag text.concat "" 4 ["a", "b", "c", "d", "e", "f", "g", "h"]`

  returns `["abcd", "bcde", "cdef", "defg", "efgh"]`. Here, `text.concat` just stands in for string concatenation.

- `swag (+) 0 3 [0, 10, 20, 30, 40, 50]`

  returns `[30, 60, 90, 120]`. Here, we are passing addition as a two-argument function.

- `swag dict.merge {} 2 [d1, d2, d3, d4]`

  returns `[{'a': 3, 'b': 2}, {'a': 3, 'b': 4, 'c':0}, {'b': 4, 'c':0}]`. Here, `d1`, `d2`, `d3`, and `d4` are dictionaries: `{'a': 10, 'b': 1}`, `{'a': 3, 'b': 2}`, `{'b': 4, 'c': 0}`, and `{}`. The function `merge(u, v)` merges two dictionaries using the key from the second argument if `u` and `v` share that key.

Questions. Include the following in comments on your GitHub pull request.

1. Briefly describe your approach and results.
2. Briefly give your answers to any of the exercises that you tackled.
3. Describe how much time you spent on this assignment.

Requirements.

☐ Implement the various versions of `swag` in Tasks #1–4 in your language of choice.

☐ Write a program that applies your function to a selection of examples (or tests) and ensures that the results are correct.

☐ Assess (at least roughly) the number of calls to the binary operation in terms of the sequence length. How does it depend on the window size?

☐ If we have discussed testing, supply a test suite for your functions and ensure all tests pass.